

Dokumentace Semantycka-Syntaktucka analiza:

Dokument je pokračováním dokumentace **lexikální analýzy**. **Syntaktická analýza** využívá **lexikální analyzátor** ke čtení vstupu z textového souboru:

```
String filenameDefault = "testInput.txt";
```

Taky lze zadat vlastní soubor pro analýzu pomocí parametru konstruktoru.

Zkratky:

F(A) - **first** pravidla A

FL(A) - **follow** pravidla A

Použití:

1. Zadání vstupu do **testInput.txt**

2. Spustit **SyntaxSemant.java**

Pro čtení vstupu z konzole používat funkci READLINE(), funkce v jazyce jsou volány v cestě kódu mezi klíčovými slovy BEGIN a END, a musí být přiřazeny nějaké proměnné, i když nic nevracejí například:

x := READLINE()

x := WRITELINE(x)

Gramatika:

G = (N, T, P, Start)

N = {Start, DecPart, CodePart, Declist, Declnit,

D, StmtList, Stmt, Assigh, Ifstm, A,

ArgList, ElsePart, V, P, Q, R, K, L, M }

T = {VAR, BEGIN, END, IF, ELSE, ELIF, ID, NUM, ;, :=, (,), ==, <=, >=, !=, <, >, {, }, +, -, *, /}

Jazyk Syntaxe (pravidla P):

Start -> DecPart CodePart	1					
DecPart -> VAR ID Declnit; DecPart eps		2	3			
Declnit -> = NUM ε	6	7				
CodePart -> BEGIN D END	8					
D -> StmtList	9					
StmtList -> Stmt StmtList ε	10	11				
Stmt -> Assigh Ifstm	12	13				
Assigh -> ID A	14					
A -> := V ; (ArgList) ;	15	16				
ArgList -> V eps	17	18				
IfStm -> IF (P) { D } ElsePart	19					
ElsePart -> ELIF (P) { D } ElsePart ELSE { D } eps		20	21	22		
P -> V Q V	23					
Q -> == <= != >= < >	24	25	26	27	28	29
V $\rightarrow$ RK	30					
R $\rightarrow$ LM	31					
K $\rightarrow$ +RK -RK ε	32	33	34			
L $\rightarrow$ (V) ID Param NUM		35	36	37		
Param -> (ArgList) ε	41	42				
M $\rightarrow$ *LM /LM ε	38	39	40			

Pro syntaktickou analýzu byla využita metoda rekurzivního sestupu popsaná ve skriptech. Pro pokračování ve vývoji programu je nejprve potřeba ověřit, zda je gramatika typu LL(1), a určit její porovnávací množiny.

Overeni na LL(1)

$FL(Start) = \{\$ \}$

$FL(Codepart) = \{\$ \}$

$FL(Decpart) = \{BEGIN\}$

$FL(Declnit) = \{;\}$

$FL(D) = \{END, \}$

$FL(Stmt) = \{ID, IF, END, \}$

$FL(Smtlist) = \{END, \}$

$FL(Assigh) = \{ID, IF, END, \}$

$FL(Ifstm) = \{ID, IF, END, \}$

$FL(ArgList) = \{ \}$

$FL(ElsePart) = \{ID, IF, END, \}$

$FL(A) = \{ID, IF, END, \}$

$FL(P) = \{ \}$

$FL(D) = \{ \}$

$FL(V) = \{;, \), ==, <=, !=, >=, <, >\}$

$FL(Q) = \{(, ID, NUM\}$

$FL(R) = \{+, -, \), ==, <=, !=, >=, <, >, ;\}$

$FL(K) = \{ \), ==, <=, !=, >=, <, >, ;\}$

$FL(L) = \{*, /, +, -, \), ==, <=, !=, >=, <, >, ;\}$

$FL(M) = \{+, -, \), ==, <=, !=, >=, <, >, ;\}$

$FL(Param) = \{*, /, +, -, \), ==, <=, !=, >=, <, >, ;\}$

Porovnavaci mnoziny

Pro kazde pravidlo $A \rightarrow b$ najdeme $F(b * FL(A))$ pro kazde pravidlo z jazyka a ocislujeme to.

{VAR} 1

{VAR} | {BEGIN} 2 3

{=} | {;} 6 7

{BEGIN} 8

{ID, IF, END, } 9

$F(\text{StmtList}) = \{F(\text{Stmt})\}$

$F(\text{Stmt}) = \{ID, IF\}$

{ID, IF} | {END, } 10 11

{ID} | {IF} 12 13

{ID} 14

{:=} | { (} 15 16

{(, ID, NUM} | {) } 17 18

{IF} 19

{ELIF} | {ELSE} | {ID, IF, END, } 20 21 22

{(, ID, NUM} 23

{==} | {<=} | {!=} | {>=} | {<} | {>} 24-29

{(, ID NUM} 30

$F(R) = \{F(L)\}$

$F(L) = \{(, ID NUM\}$

{(, ID NUM} 31

{+} | {-} | { }, ==, <=, !=, >=, <, >, ; } 32 33 34

{ } | {ID} | {Num} 35 36 37

{ } | { *, /, +, -,), ==, <=, !=, >=, <, >, ; } 41 42

{*} | {/} | {+, -,), ==, <=, !=, >=, <, >, ; } 38 39 40

Implementovaný překladač využívá jednoduchý jednopřechodový analyzátor. Součástí syntaktické analýzy je také semantická analýza, která zajišťuje kontrolu sémantické správnosti programu, rozpoznávání funkcí a vyhodnocování výrazů.

Semantická pravidla:

Start $\rightarrow$ DecPart CodePart {Func.add("READLINE", "WRITELINE")}

DecPart $\rightarrow$ VAR ID Declnit; DecPart | eps

{If ID in Sym then error else ID to table Sym; Declnit.id = ID}

Declnit $\rightarrow$ = NUM | eps

CodePart $\rightarrow$ BEGIN D END

D $\rightarrow$ StmtList

StmtList $\rightarrow$ Stmt StmtList | ϵ

Stmt $\rightarrow$ Assign | Ifstm

Assign $\rightarrow$ ID A {If ID in Sym or func; A.id = ID else error}

A $\rightarrow$:= V ; | (ArgList) ;

{If A.id in Sym : A.id.val = V.val; (id ; val) to table Sym; id to table Init}

{If A.id in Func Then error; A.val = ArgList.val}

ArgList $\rightarrow$ V | eps {ArgList.val = V.val}

IfStm $\rightarrow$ IF (P) { D } ElsePart

ElsePart $\rightarrow$ ELIF (P) { D } ElsePart | ELSE { D } | eps

P $\rightarrow$ V Q V {P.val = V.val Q.operator V.val}

Q $\rightarrow$ == | <= | != | >= | < | > {Q.operator = S_EEQ}...{Q.operator = S_G}

V $\rightarrow$ RK { K.in = R.val } { V.val = K.out }

R $\rightarrow$ LM { M.in = L.val; R.val = M.out }

K $\rightarrow$ +RK | -RK | ϵ

{ K1.in = K.in + R.val; K.out = K1.out }

{ K1.in = K.in - R.val; K.out = K1.out }

{ K.out = K.in }

L $\rightarrow$ (V) | ID | NUM

{L.val = V.val}

{If ID in Sym && ID in Init then L.val = Sym.getValueFromKey(ID)}

{L.val = NUM}

$M \rightarrow *LM \mid /LM \mid \varepsilon$

{ M1.in = M.in * L.val; M.out = M1.out }

{ M1.in = M.in / L.val; M.out = M1.out }

{ M.out = M.in }

Param $\rightarrow$ (ArgList) $\mid \varepsilon$

{Param.out = ArgList.val}